From PDFs to a Skill Tree: A Bayesian Adaptive Mastery System for Arbitrary Course Material

Goh Cheng Arn David[1]

Department of Computer Science, University of Toronto
daveed@cs.toronto.edu

Preprint, June 2026. The 3-page short version was accepted at AIED 2026 Interactive Events (submission #2065; presented July 2, 2026). Hosted demo: https://quizvid.vercel.app. Source: https://github.com/davidcagoh/adaptive-learner.

Abstract. Khan Academy-style mastery learning is effective but hand-authored, and exists for at most a few dozen canonical courses. We describe a working system that generates a Khan-Academy-style mastery experience from arbitrary text-based course material in minutes, and uses a Bayesian adaptive diagnostic to place a learner onto the resulting skill tree before the mastery loop begins. The system contributes (i) a deterministic-with-LLM-fallback pipeline that emits a self-contained offline mastery app from PDFs in a single bring-your-own-key run, (ii) a domain-agnostic Bayesian engine extended for prerequisite-DAG placement via a graph-Laplacian prior, a frontier-stopping criterion, and an asymmetric credential rule, and (iii) a design choice that renders the diagnostic onto the skill tree itself rather than as a separate summary screen. The hosted demo lets a reviewer run the full path in roughly five minutes; three pre-authored courses are available for inspection without uploading.

Keywords: adaptive testing · mastery learning · Bayesian inference · LLM-generated curricula · skill trees

1 Introduction

Khan Academy-style mastery learning works: a learner who reaches 100% on a graph of prerequisite-linked skills, with adaptive practice and immediate corrective feedback at each node, outperforms equivalent learners under matched conditions [2,9,10]. The mechanism does not generalize. A mastery system for a single curriculum (introductory algebra, AP Biology) takes editor-years to author — a hand-curated DAG of skills, a calibrated question bank at each node, and human review of every prerequisite edge — and exists for at most a few dozen canonical courses. Outside that envelope, the same mechanism is unavailable. We describe a working system that generates a Khan-Academy-style mastery

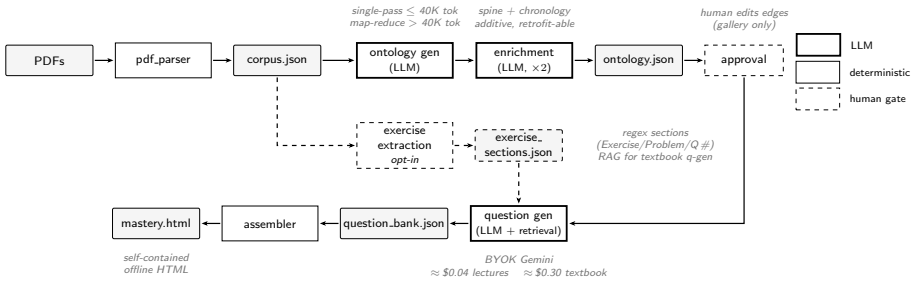

Fig. 1. Pipeline. Ingest produces a structured corpus (deterministic). LLM stages (thick border) generate the prerequisite DAG and a per-node question bank. Exercise extraction is opt-in and feeds question generation as RAG context for textbook corpora. Approval is a human gate applied only to gallery courses; user-uploaded courses skip it. Cost is paid in the user's BYOK Gemini budget.

experience from arbitrary course material — lecture PDFs, slides, lecture notes — in minutes (Fig. 1), and that uses an adaptive Bayesian diagnostic to place a learner onto the resulting skill tree before the mastery loop begins (Fig. 2). The hosted demo lets a reviewer run the full path — upload PDFs, watch a skill tree build, take placement, work the mastery loop — at the demo URL in roughly five minutes.

The pipeline and the diagnostic engine are independently useful: the pipeline emits a self-contained mastery app consumable without the diagnostic, and the engine is domain-agnostic — it can be applied to any assessment domain by supplying a question bank and a domain-appropriate prior. They meet at deploy time through the assembled HTML. Section 2 details both. Section 3 walks through the demonstration at `https://quizvid.vercel.app`. Section 4 covers deployment and limitations; Section 5 closes with what is next.

2 Architecture

2.1 Pipeline (corpus → ontology + question bank)

The pipeline takes an arbitrary corpus — typically PDFs of lecture slides, notes, or a textbook — and produces a course-specific skill ontology and a typed question bank against it. Parsing is deterministic on the common path: a fast text-layer extractor feeds rules-based table-of-contents and heading detection that infers page-number offsets and extracts section boundaries. When rules-based detection produces sparse output, or when the PDF has no text layer at all, the system sends the original PDF directly to a multimodal LLM as a single bounded call; the model returns a structured corpus JSON with sections and headings, internally handling OCR and unusual layouts. There is no recursive agent loop and no separate OCR stage to maintain. The fallback cost is paid in the user's own LLM budget under the project's bring-your-own-key model. The resulting structured

corpus is fed to an ontology generator that auto-selects between single-pass and map-reduce strategies based on token volume; map-reduce uses a per-document chunking pass, an LLM-driven consolidation pass that merges granular nodes into broader concepts, and a deterministic node-budget trim as a fallback. A cheap upstream planning call derives the target node count from the corpus structure rather than fixing it. Two further LLM calls then enrich the ontology with a per-topic spine (the through-line concepts of each branch) and a topic-to-source-document chronology used to order branches left-to-right in the rendered tree; both fields are additive and non-fatal on failure, allowing a graceful fallback to topology-based ordering.

For textbook corpora, a deterministic regex-based pass scans the corpus for exercise sections and feeds the matched text per-node into the question-generation prompt as RAG-style context [5].[1] The ontology is checked against schema validity (DAG, ≥ 6 questions per node, valid topological sort) and a $\geq 70\%$ structural-element coverage check whose denominator adapts to the corpus shape (input documents, TOC entries, or detected headings). User-uploaded courses receive the automated checks and surface any warnings rather than being rejected.

2.2 Adaptive Diagnostic

Placement on a prerequisite DAG is the system's most novel component. Knowledge tracing literature (BKT [3] and its modern extensions [4,1], including graph-based variants [7]) typically targets per-skill mastery probability across a long sequence of attempts; we instead want *cold-start placement* on a fresh course in tens of questions. We build on a domain-agnostic Bayesian adaptive assessment engine whose core is a Gaussian state-space model with a rank-1 Kalman update. The latent state $\boldsymbol{\theta} \in \mathbb{R}^n$ assigns one dimension per skill node; $\theta_i > 0$ encodes the belief that the learner has acquired node i. For a question targeting node j with weight vector $\mathbf{w}$ ($w_j = 1$, zeros elsewhere) and binary response $y \in \{-1, +1\}$ (incorrect / correct), the observation model is

$$y \mid \boldsymbol{\theta} \;\sim\; \mathcal{N}(\mathbf{w}^\top \boldsymbol{\theta},\, \sigma^2), \qquad \sigma^2 = 1.0, \tag{1}$$

and the posterior is updated by a rank-1 Kalman step:

$$\mathbf{K} = \frac{\Sigma \mathbf{w}}{\mathbf{w}^\top \Sigma \mathbf{w} + \sigma^2}, \quad \boldsymbol{\mu}' = \boldsymbol{\mu} + \mathbf{K}(y - \mathbf{w}^\top \boldsymbol{\mu}), \quad \Sigma' = \Sigma - \mathbf{K}\mathbf{w}^\top \Sigma. \tag{2}$$

Applying this engine to a 156-node prerequisite DAG exposed three structural mismatches with a default isotropic instantiation: skill dimensions are not independent, the latent variable of interest is "what node can the learner unlock next" rather than a scalar level, and a multiple-choice question carries a non-trivial guess probability that a symmetric Gaussian likelihood does not adequately model.

[1] Opt-in: runs as a separate pipeline stage so slide-deck corpora do not pay the scan cost.

We address each mismatch with a small, principled change rather than a domain-specific engine fork. **(1) DAG-aware prior.** Earlier work in the same engine encoded prerequisite correlation in the per-question weight matrix W (w[primary] = 1.0, w[direct prereq] = 0.3). We move that structure into the prior covariance:

$$\Sigma_0 = (I + \lambda L)^{-1}, \tag{3}$$

where L is the normalized graph Laplacian of the prerequisite DAG [8] and $\lambda = 0.3$ controls prereq-correlation strength ($\lambda = 0$ recovers an isotropic prior; larger λ couples node estimates more tightly and can slow convergence on deep DAGs). The value $\lambda = 0.3$ was selected by comparing posterior uncertainty trajectories on held-out questions from two courses (csc2515 and MAT1856). Connected nodes start positively correlated; transitive prerequisites — which the flat-0.3 per-question weight scheme ignored — receive correlation weight that decays naturally with graph distance. The engine accepts a `Prior` object at construction; domains with independent dimensions pass the default `IdentityPrior()` and are unaffected. **(2) Frontier stopping.** A learner using a 156-node skill tree does not need confident μ on every node — they need confident knowledge of which nodes they can next unlock. We define the frontier F as the set of unmastered nodes whose prerequisites are all mastered. Treating diagonal posterior entries as independent, the probability that node i is frontier-eligible is

$$p_i = \Phi\left(\frac{-\mu_i}{\sqrt{\Sigma_{ii}}}\right) \prod_{j \in \mathrm{pre}(i)} \Phi\left(\frac{\mu_j}{\sqrt{\Sigma_{jj}}}\right), \tag{4}$$

where Φ is the standard normal CDF. We stop when $\forall i:\ p_i \geq \hat{p}$ or $p_i \leq 1 - \hat{p}$, with $\hat{p} = 0.9$. This aligns the stop condition with the user-visible output (the rendered tree) and keeps the diagnostic short on large courses. **(3) Asymmetric credential rule.** Rather than tune a per-question σ^2 to model guess probability, we separate the placement *decision* from the engine posterior. A node is credited as mastered iff the learner answered ≥ 2 questions on that node correctly with zero wrong answers; the posterior μ is used only for question selection ($\arg\max_i w_i^\top \Sigma w_i$), never as the placement criterion. The mechanism is asymmetric in the right direction: when a learner answers a node-N question correctly for the first time, the engine records the question (`mark_asked`) but applies *no* Bayesian update — the selector enqueues a second N question, which alone decides credit. A wrong answer at any point fires a normal $y = -1$ update, discards the tentative correct, and disqualifies N permanently for the session. Slip-by-luck on the first answer is exposed cheaply; an honest correct learner is verified in two questions. We further restrict the diagnostic question pool to MCQ only: true/false has a 1/2 baseline guess rate that produced unacceptable false-positive masteries in early calibration runs, while 4-option MCQ at 1/4 baseline brings the simulated pure-guesser false-positive rate to $\approx 5\%$ on a 20-question budget across both Python and JavaScript implementations.

Placement is binary (mastered / not-yet) rather than multi-level: the rendered tree only shows mastered, available, and locked states, and a multi-bucket scheme

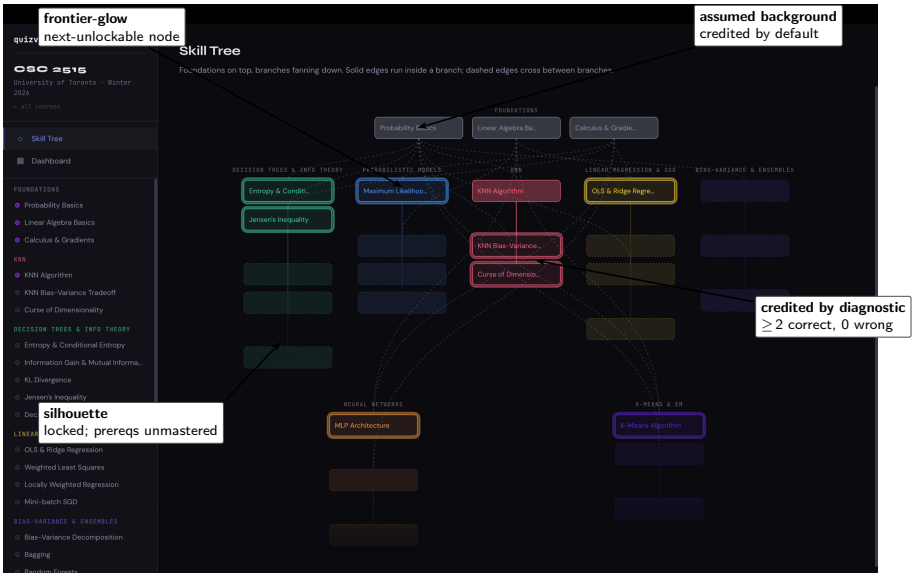

Fig. 2. Skill tree mid-session, csc2515 ML lectures. The diagnostic places the learner onto the tree itself rather than a separate summary screen. **Frontier-glow** marks next-unlockable nodes; **silhouettes** mark locked nodes whose prerequisites are not yet mastered; the foundations row is credited as **assumed background**; the KNN cluster is **credited by diagnostic** (≥ 2 correct, 0 wrong). Branch order left-to-right reflects course chronology, derived from a topic-to-source-document mapping (§2.1).

adds an ambiguous stopping criterion without informing any user-visible decision. Placement is additive only — re-running the diagnostic cannot demote a learner who has progressed. The credential rule is intentionally conservative: on a 28-node course with a 20-question budget, an honest knows-everything learner is credited with at most $\sim$10 nodes; the remainder enters the regular mastery loop, which is the conventional path to mastery in this framework [3].

3 Demonstration

The skill tree is a single UI surface that unifies four moments: ontology approval, diagnostic placement, mastery-loop frontier, and standalone artifact. State rendering follows three principles drawn from skill-tree conventions in games: aspiration over overwhelm (locked nodes render as silhouettes, not blacked-out), frontier inversion (the *next-unlockable* node carries the visual emphasis, not the just-mastered one), and conservative claims (off-spine nodes render identically to spine nodes because spine identification is a coarse signal we will not over-trust). The diagnostic placement is itself rendered onto this tree — the diagnostic does not have a separate summary screen — which collapses two surfaces into one and makes the learner's first contact with the tree already personalised.

The demo at `https://quizvid.vercel.app` is structured as a hybrid surface, mirroring the deployment shape (§4). The landing page presents two real artifacts above the fold — a gallery of three deliberately-chosen courses (csc2515 ML lectures, cg3207 computer architecture with diagrams and Verilog, MAT1856 mathematical finance with dense formula notation) and an upload zone for custom corpora — without a marketing hero. The three courses span the pipeline's working envelope of text-based STEM with explicit prerequisite structure. A reviewer reaches a working mastery loop in two clicks: click a course card, click a node on the rendered tree, answer a question, watch the frontier move. The placement modal that appears on first visit is dismissible; reviewers who want to inspect the tree directly take "Skip placement, browse the tree." Self-studiers take placement, which seeds a personalised frontier in 15–25 questions.

4 Deployment and Limitations

The deployed system has two surfaces, mirroring the landing page split. The curated gallery — three pre-authored courses, picked to span the pipeline's working envelope rather than to maximise count — is served as static assets; no server, no database, no per-user cost. A learner's progress lives in browser `localStorage`; the diagnostic and mastery engine run entirely client-side via a closed-form Kalman update. Course authoring is offered as a thin BYOK web form: the user supplies their own LLM API key, uploads a corpus, and the pipeline runs on serverless compute and returns a generated course. We do not store user keys, do not host user-generated courses, and do not require sign-in.

Cost transparency is enforced before any API call: the system reports input tokens, auto-selected strategy (single-pass vs. map-reduce, switched at 40K tokens), and a dollar estimate, and waits for explicit confirmation. A typical lecture-set corpus generates a course for under five cents in API spend; a textbook-sized corpus costs roughly thirty cents.

Limitations are direct. The method is scoped to text-based STEM curricula with an explicit prerequisite structure; humanities, project-based, and discussion-driven courses are out of scope for this version because the pipeline's prerequisite-DAG extraction and the question-bank generator (multiple-choice, true/false, short derivation) presume discrete factual content. The curated gallery is three courses rather than six in order to stay inside that scope honestly. The pipeline's output quality is bounded by current LLM consistency; we mitigate via deterministic node-budget enforcement and a human approval step on the generated ontology. The BYOK requirement keeps custom course authoring restricted to users willing to obtain and paste an API key. Image-only PDFs (scanned textbooks, handwritten notes) need OCR before entering the pipeline; this is flagged inline in the upload-zone format guidance rather than handled silently. Cross-course transfer of mastery state, hosted progress storage, and richer noise modelling for adversarial responses are explicit future work.

5 Conclusion and Future Work

The system covers the path from arbitrary course PDFs to an adaptive mastery app a learner can run offline, and from cold-start to a personalised starting frontier in roughly twenty multiple-choice questions. Cross-course mastery transfer is currently disabled because ontologies are independently generated and identifier collisions are coincidental; resolving this needs a shared concept layer the pipeline does not yet emit. Richer noise modelling — partial credit, time-on-question as a side signal [6] — is the obvious next research direction. Closer in, we want telemetry from real BYOK users to learn whether the conservative credential rule leaves more nodes uncredited than is comfortable, and whether a hybrid rule combining credential evidence with the Laplacian-propagated posterior is worth the extra tunable knob.

References

1. Abdelrahman, G., Wang, Q., Nunes, B.P.: Knowledge tracing: A survey. ACM Computing Surveys **55**(11), Article 224 (2023). https://doi.org/10.1145/3569576
2. Bloom, B.S.: The 2 sigma problem: The search for methods of group instruction as effective as one-to-one tutoring. Educational Researcher **13**(6), 4–16 (1984). https://doi.org/10.3102/0013189X013006004
3. Corbett, A.T., Anderson, J.R.: Knowledge tracing: Modeling the acquisition of procedural knowledge. User Modeling and User-Adapted Interaction **4**(4), 253–278 (1995). https://doi.org/10.1007/BF01099821
4. Ghosh, A., Heffernan, N., Lan, A.S.: Context-aware attentive knowledge tracing. In: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD 2020). pp. 2330–2339 (2020). https://doi.org/10.1145/3394486.3403282
5. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems 33 (NeurIPS 2020). pp. 9459–9474 (2020). https://doi.org/10.48550/arXiv.2005.11401
6. van der Linden, W.J., Glas, C.A.W. (eds.): Elements of Adaptive Testing. Statistics for Social and Behavioral Sciences, Springer, New York (2010). https://doi.org/10.1007/978-0-387-85461-8
7. Nakagawa, H., Iwasawa, Y., Matsuo, Y.: Graph-based knowledge tracing: Modeling student proficiency using graph neural networks. In: 2019 IEEE/WIC/ACM International Conference on Web Intelligence (WI 2019). pp. 156–163 (2019). https://doi.org/10.1145/3350546.3352513
8. Smola, A.J., Kondor, R.: Kernels and regularization on graphs. In: Learning Theory and Kernel Machines (COLT/Kernel 2003). Lecture Notes in Computer Science, vol. 2777, pp. 144–158. Springer (2003). https://doi.org/10.1007/978-3-540-45167-9_12
9. VanLehn, K.: The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. Educational Psychologist **46**(4), 197–221 (2011). https://doi.org/10.1080/00461520.2011.611369

10. Wang, S., Christensen, C., Cui, W., Tong, R., Yarnall, L., Shear, L., Feng, M.: When adaptive learning is effective learning: Comparison of an adaptive learning system to alternate conditions. Interactive Learning Environments **31**(2), 793–803 (2023). https://doi.org/10.1080/10494820.2020.1808794